Reverse Engineering Half-Life

Chamod Lakshitha

Contents

1. Introduction: Half-Life 1
   1.1 Game Overview 1
   1.2 Narrative Context 1
   1.3 The GoldSrc Engine: A Legacy of Innovation 1
   1.4 Why Half-Life for Reverse Engineering? 1
2. Tools and Methodology 2
   2.1 Software Toolset 2
   2.2 Methodology Workflow 2
3. Static and Dynamic Analysis 3
   3.1 Dynamic analysis via X32dgb 3
      3.1.2 Memory Architecture and Endianness 4
   3.2 Static Analysis via Ghidra 5
   3.3 Static Analysis Outcomes 7
4. Assembly-Level Modification 8
   4.1 Identifying the Subtraction 8
   4.2 Applying the Patch (NOP) 8
   4.3 Technical Outcome 9
   4.4 Implementation of Player-Specific Logic Bypass (Return Patch) 9
   4.5 Technical Outcome 9
5. Security Discussion & Mitigation Strategies 10
   5.1 Vulnerability Analysis 10
   5.2 Mitigation Strategies 10

1. Introduction: Half-Life

1.1 Game Overview

Released in 1998 by Valve Corporation, Half-Life is a foundational title in the First-Person Shooter (FPS) genre. Unlike its contemporaries, which relied on segmented levels and cutscenes, Half-Life introduced a seamless, continuous world where the story is told entirely through the player's perspective. From a technical standpoint, it is a masterpiece of C++ modularity, utilizing a DLL-based architecture that separates the game engine from the game logic.

1.2 Narrative Context

The player assumes the role of **Dr. Gordon Freeman**, a theoretical physicist at the **Black Mesa Research Facility**. Following a catastrophic "Resonance Cascade" during a teleportation experiment, Freeman must fight his way out of the facility while navigating an alien invasion and a government cover-up.

1.3 The GoldSrc Engine: A Legacy of Innovation

The game is built on the **GoldSrc (Goldsource)** engine. Historically, this engine is a heavily modified fork of the *Quake* source code. Its importance in this security research project stems from its transparent memory management and its influence on modern game architecture.

Key Technical Features of the Game:

- **Skeletal Animation:** One of the first engines to use complex bone-based animations for NPCs.
- **Scripted Sequences:** Advanced AI triggers that allow the environment to react dynamically to the player's presence.
- **Environmental Physics:** A robust calculation system for gravity, friction, and fluid dynamics all of which interact directly with the player's health variables.

1.4 Why Half-Life for Reverse Engineering?

I. **Complexity:** It utilizes multiple inter-linking DLLs (hl.dll, client.dll, hw.dll), mimicking real-world enterprise software.

II. **Lack of Obfuscation:** While the code is compiled, it does not use the aggressive anti-tamper measures seen in modern titles (like Denuvo), allowing for a pure study of **Assembly-level logic**.

III. **Community Knowledge:** Due to the engine being partially open-sourced (via the SDK), researchers can verify their assembly findings against known C++ structures.

2. Tools and Methodology

The reverse engineering process for this research employed a dual-analysis approach, combining real-time memory monitoring with deep-code static analysis. The methodology was designed to move from surface-level display addresses to the core engine logic responsible for health calculations.

2.1 Software Toolset

The analysis was performed on a Windows 10 Virtual Machine hosted within VMware, configured as an internal server to prevent external network exposure during binary manipulation.

- **x32dbg:** A professional-grade Windows debugger used for **Dynamic Analysis**. It was utilized to attach to the running hl.exe process, perform memory pattern matching, and set hardware breakpoints on specific memory addresses to observe real-time data flow.
- **Ghidra:** A sophisticated software reverse engineering (SRE) suite used for **Static Analysis**. Ghidra allowed for the deconstruction of the hl.dll binary into human-readable via its decompiler, enabling the identification of class structures and function offsets without executing the game.

2.2 Methodology Workflow

1. **Dynamic Analysis (x32dbg):** Executed memory scans for active health values, filtering for changes during damage events to isolate the memory address responsible for the HUD display.
2. **Static Analysis (Ghidra):** Decompiled the game's core library to trace HUD strings, mapping the internal player structure and identifying the specific offset and data type used for health storage.
3. **Instruction Isolation & Initial Patch:** Located the shared arithmetic instructions responsible for health reduction. Applied a **NOP** patch to bypass the logic, which resulted in global invulnerability across all game entities.
4. **Surgical Logic Bypass**: Located the shared arithmetic instructions responsible for health reduction. Applied a **NOP** patch to bypass the logic, which resulted in global invulnerability across all game entities.

3. Static and Dynamic Analysis

3.1 Dynamic analysis via X32dgb

Initial dynamic analysis using **x32dbg** identified the memory address 001895F8 through pattern matching. However, this was identified as a **Display-Only** address (HUD). Modifying this value only changed the visual number on the screen without affecting the actual player state, indicating the "True Health" was stored elsewhere in the player entity structure.

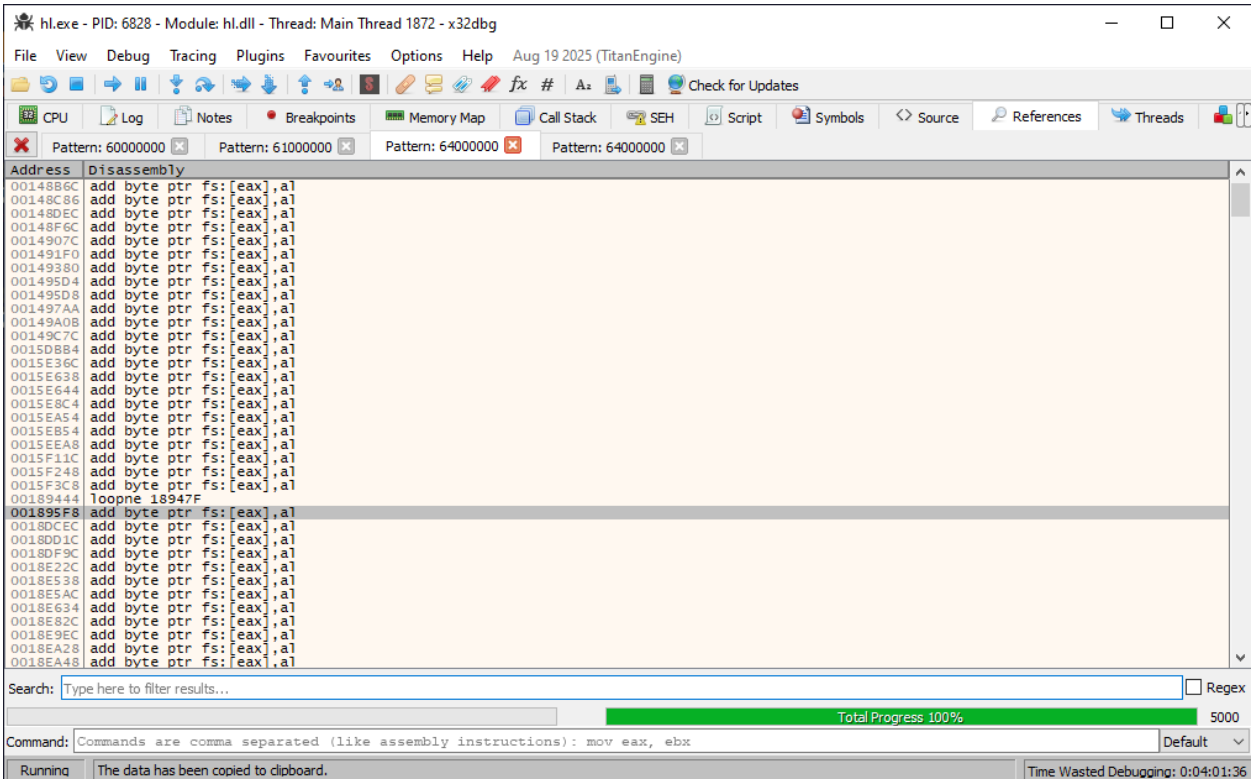

Figure 1: Initial Pattern Match at Full Health (0x64)

Description: This figure shows the initial pattern search for 64000000. Since 0x64 in hexadecimal corresponds to 100 in decimal, this scan was used to identify the memory address storing the player's starting health. The address **001895F8** was successfully isolated as the primary HUD candidate.

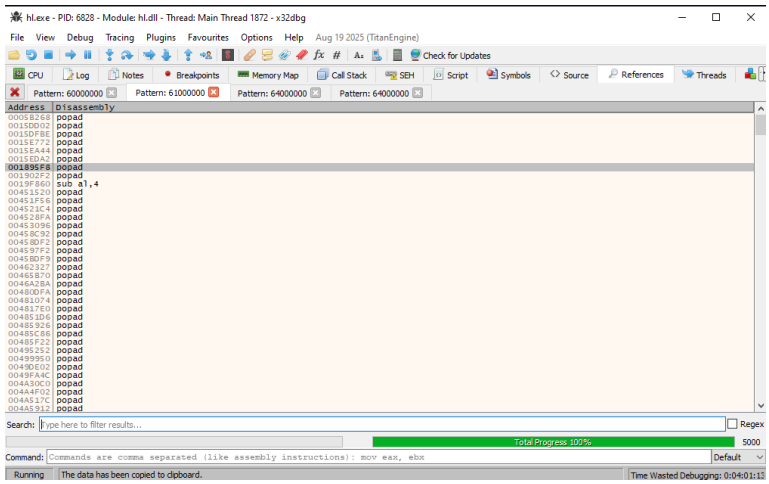

Figure 2: Pattern Match for Incremental Damage (0x61)

Description: After the player took a small amount of damage, a second scan was conducted for the pattern 61000000 (0x61 = 97 decimal). The consistent appearance of **001895F8** in the results verified that this memory location updates dynamically as the player's health status changes during gameplay.

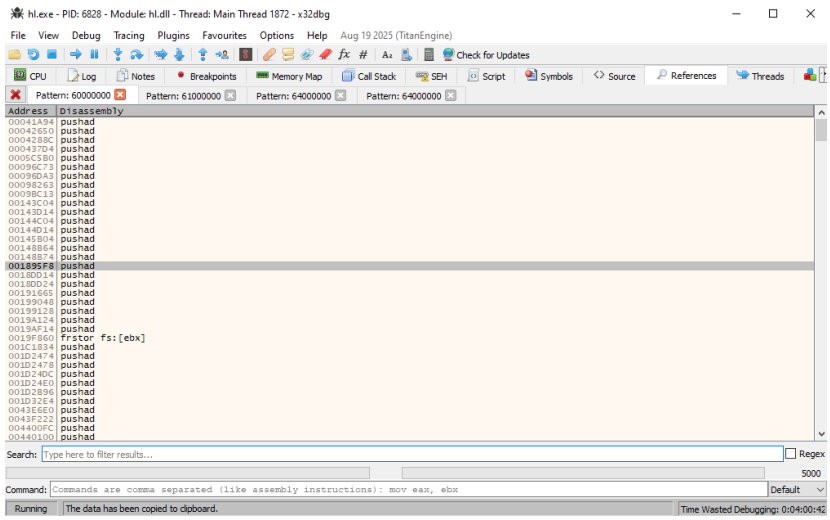

Figure 3: Static Address Verification (0x60)

Description: A final verification scan using the pattern 60000000 (0x60 = 96 decimal) confirmed that address **001895F8** remained the active pointer for health data. By tracking the hex-to-decimal decrement across these three figures, it was definitively proven that this address serves as the HUD's health display anchor.

3.1.2 Memory Architecture and Endianness

Dynamic analysis confirmed that health values utilize Little-Endian formatting. A search for decimal 100 used the hex string 64 00 00 00, placing the least significant byte first. This architectural alignment was essential for pattern matching and verified that address 001895F8 is a raw memory pointer rather than an obfuscated variable.

3.2 Static Analysis via Ghidra

To find the internal logic, the **hl.dll** was loaded into Ghidra.

1. **Format String Identification:** Searched for the HUD display function FUN_10061c80, which used the string "*2 Health: %i2%%*". This confirmed that health is passed to HUD from an underlying structure.

```
if (iVar6 == 2) {
  aiStack_170[1] = (*DAT_100d629c)(*(undefined4 *)(piVar8[1] + 0x208));
  iVar6 = 0;
  do {
    bVar1 = s_1_%p1_2_Health:_%i2%%_3_Armor:_%_100cc7a0[iVar6];
    abStack_104[iVar6] = bVar1;
    iVar6 = iVar6 + 1;
  } while (bVar1 != 0);
  iVar6 = (**(code **)(*DAT_100d6134 + 200))(param_1,piVar8);
```

Figure 4: Decompiled view (Ghidra) of the function FUN_10061c80

2. **Entity Death Logic:** Located *CBaseEntity::SUB_Remove*. Analysis showed that upon entity removal/death, the memory at *base_address + 0x160* was zeroed out: **(undefined4 *)(*(int *)(this + 4) + 0x160) = 0;*

```
Decompile: SUB_Remove - (hl.dll)

/* public: void __thiscall CBaseEntity::SUB_Remove(void) */

void __thiscall CBaseEntity::SUB_Remove(CBaseEntity *this)

{
                    /* 0x7bcf0  171  ?SUB_Remove@CBaseEntity@@QAEXXZ */
  FUN_1007bc80((int)this);
  if (0.0 < *(float *)(*(int *)(this + 4) + 0x160)) {
    *(undefined4 *)(*(int *)(this + 4) + 0x160) = 0;
    (*DAT_100d6274)(2,s_SUB_Remove_called_on_entity_with_100cf080);
  }
  (*DAT_100d61d8)(*(undefined4 *)(*(int *)(this + 4) + 0x208));
  return;
}
```

Figure 6: Decompiled view (Ghidra) of Sub_Remove

3. **Normalization Confirmation:** Analyzed the monster initialization (*monster_flyer*). The engine sets the same offset (+0x160) to 0x3f800000. In IEEE-754 floating-point representation, this hex value equals 1.0, confirming health is stored as a float where 1.0 represents 100%.

```
Decompile: FUN_10001710 - (hl.dll)

void __fastcall FUN_10001710(int param_1)

{
  *(undefined4 *)(*(int *)(param_1 + 4) + 0x170) = 0;
  **(int **)(param_1 + 4) = (int)s_monster_flyer_100bc2d4 - *(int *)(DAT_100d63f4 + 0x98);
  *(undefined4 *)(*(int *)(param_1 + 4) + 0x10c) = 3;
  *(undefined4 *)(*(int *)(param_1 + 4) + 0x108) = 5;
  *(undefined4 *)(*(int *)(param_1 + 4) + 0x16c) = 0;
  *(undefined4 *)(*(int *)(param_1 + 4) + 0x160) = 0x3f800000;
  *(undefined4 *)(param_1 + 0x2a4) = 0;
  *(undefined4 *)(param_1 + 0x154) = 0x3e4ccccd;
  (*DAT_100d6188)(*(undefined4 *)(*(int *)(param_1 + 4) + 0x208),s_models/boid.mdl_100bc260);
  FUN_10089900();
  return;
}
```

Figure 7: Decompiled view (Ghidra) of function FUN_10001710

4. **Entry Point for Player Specific TakeDamage:** Traced the references of the 0x160 offset to isolate the core damage-processing logic. It is the first function called when the player takes damage. It handles player-specific logic like armor reduction and HEV suit alerts before passing the actual damage value down to the core damage function FUN_10018220 at 10018220.

```
if ((param_5 & 0x21) != 0) {
  if (bVar2) {
    pcVar9 = s_!HEV_DMG5_100cc710;
  }
  else {
    pcVar9 = s_!HEV_DMG4_100cc71c;
  }
  FUN_100626d0(param_1,(byte *)pcVar9,0,0x1e);
  param_5 = param_5 & 0xffffffde;
  bVar3 = true;
}
if ((param_5 & 2) != 0) {
  if (5 < param_1[0x10e]) {
    FUN_100626d0(param_1,(byte *)s_!HEV_DMG6_100cc704,0,0x1e);
```

Figure 8: Decompiled view (Ghidra) of identified Take Damage function

5. **Damage Executor Function:** This performs the actual health subtraction via the FSUB instruction at 10018551 and handles the death check. This function is shared across multiple entity types, not just the player.

Figure 9: Listed & Decompiled View (Ghidra) of the damage executor function

3.3 Static Analysis Outcomes

- **Data Structure Mapping:** Identified the internal entity structure, confirming health is stored at **offset 0x160**.
- **Data Type Validation:** Verified health is represented as a 32-bit normalized float (1.0 = 100%) rather than a standard integer.
- **Logic Identification:** Located the global damage-processing instruction (FSUB) at 10018551.
- **Polymorphic Discovery:** Observed that the core damage logic is shared across all entities, explaining why the initial NOP patch caused "Global God Mode."
- **Function Isolation:** Identified the player-exclusive function *CBasePlayer::TakeDamage* at hl.dll+1005fa70 via vtable analysis, enabling a surgical patch.

4. Assembly-Level Modification

4.1 Identifying the Subtraction

By searching for the *0x160* offset and filtering for floating-point subtraction instructions (**FSUB**), function **10018551** was identified as the primary damage handler.

The Target Instruction

The following assembly instruction was identified as the culprit for decreasing player health:

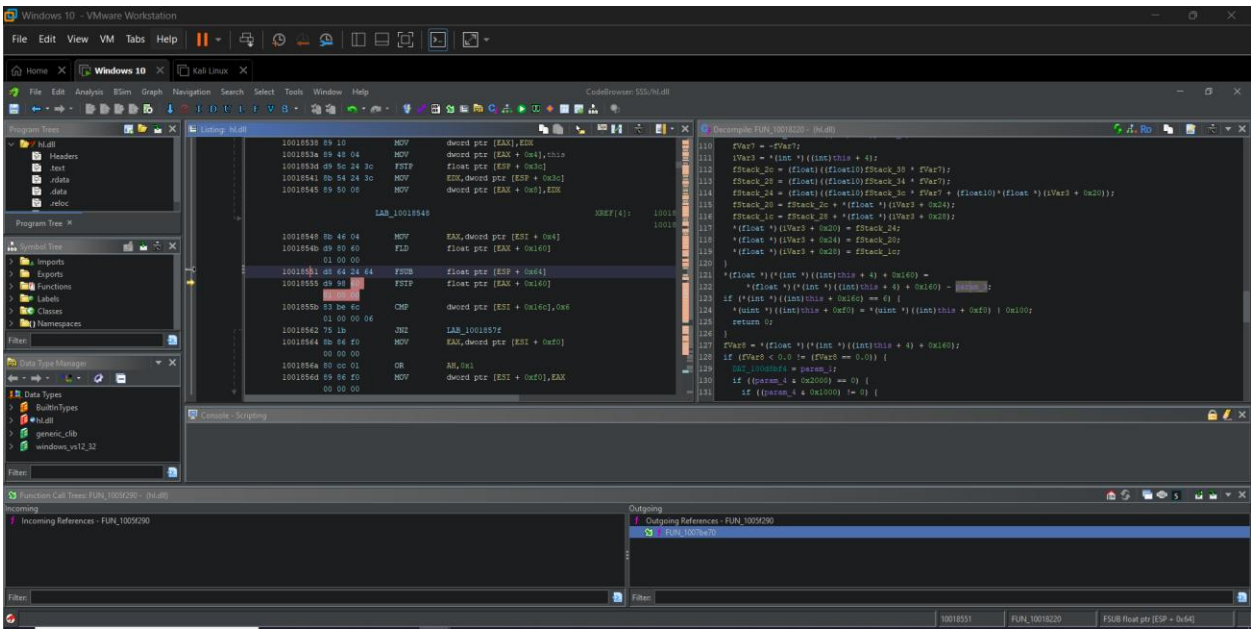

Figure 10: The function identified as the primary damage handler

```
1D038551    D86424 64    fsub dword ptr ss:[esp+64]
1D038555    D998 60010000    fstp dword ptr ds:[eax+160]
```

Figure 11: CPU view (x32dgb) of Instructions responsible for the subtraction

4.2 Applying the Patch (NOP)

Using x32dbg, the instruction at hl.dll+18551 was replaced with a series of NOP (No-Operation) instructions (0x90).

```
0FEB8551    90    nop
0FEB8552    90    nop
0FEB8553    90    nop
0FEB8554    90    nop
0FEB8555    90    nop
0FEB8556    90    nop
0FEB8557    90    nop
0FEB8558    90    nop
0FEB8559    90    nop
0FEB855A    90    nop
```

Figure 12: CPU view (x32dbg) of those instructions after patching them with NOP

4.3 Technical Outcome

By neutralizing these instructions (applying NOP), the CPU is effectively instructed to bypass the floating-point subtraction that calculates health reduction.

Problem Statement: While this patch successfully achieved the goal, it resulted in a "**Global God Mode**" effect where all NPCs (monsters and scientists) also became invincible. The engine utilizes this shared code path for all entity damage calculations, every NPC, highlighting the lack of entity-specific filtering in the initial logic bypass.

Solution: The patch was applied at 1005fa70 *(CBasePlayer::TakeDamage)* in x32dbg by replacing the function entry point with two instructions.

4.4 Implementation of Player-Specific Logic Bypass (Return Patch)

To resolve the entity-filtering issues of the previous NOP patch, the modification was moved to the function start at 1005fa70. This address corresponds to the *CBasePlayer::TakeDamage* function, which the engine's vtable system calls **exclusively for the player entity**.

```
                    LAB_1005fa70                                XREF[1]:
1005fa70 83 ec 08        SUB        ESP,0x8
1005fa73 53              PUSH       EBX
```

Figure 13: Listed View (Ghidra) of the two logic instructions

```
1D07FA70   83EC 08   sub esp,8
1D07FA73   53        push ebx
```

Figure 14: CPU View (x32dbg) of the two logic instructions

The entry logic was overwritten with two specific instructions:

I. **MOV EAX, 1**: This sets the return register to 1, spoofing a "successful" execution signal to the calling engine logic.
II. **RET 0x10:** This instruction immediately cleans the stack and exits the function.

```
1D07FA70   B8 01000000   mov eax,1
1D07FA75   C2 1000       ret 10
```

Figure 15: CPU View (x32dgb) of the entry logic after it was overwritten with two specific instructions

4.5 Technical Outcome

By forcing an immediate return, the CPU exits the function before any health-processing logic or subtraction can occur. Because this function is unique to the player class, NPCs and monsters which utilize separate damage functions remain unaffected and continue to take damage normally. This results in a surgical, player-only invulnerability that is more efficient and reliable than a mid-function NOP.

5. Security Discussion & Mitigation Strategies

The successful implementation of this patch highlights a significant vulnerability in client-side trust. Because the game engine relies on local binaries (hl.dll) to process critical logic like damage and health, an attacker with memory-write access can bypass server-side expectations.

5.1 Vulnerability Analysis

- **Logic Hijacking:** By overwriting the function entry point with RET 0x10, the program's control flow is diverted before security checks or state updates can occur.
- **Lack of Integrity Checks:** The application fails to verify the cryptographic hash of its loaded modules. This allows persistent modifications to the instruction pointer without triggering a crash or alert.
- **Client-Side Authority:** The engine processes health reduction locally. In a secure architecture, the server should act as the "Source of Truth," calculating damage and pushing state updates to the client.

5.2 Mitigation Strategies

To prevent this type of reverse engineering and patching, the following controls should be implemented:

- **Module Integrity Validation:** Implement **checksum verification** (CRC32 or SHA-256) at runtime. If the hash of hl.dll in memory differs from the original disk version, the application should terminate.
- **Server-Side Validation:** Move health and damage logic to the server. The client should only send "intent" (I fired a shot"), while the server calculates the result, preventing local RET or NOP patches from affecting game state.
- **Anti-Debugging & Obfuscation:** Utilize **code packing** and **anti-attach** techniques to prevent debuggers like x32dbg from attaching to the process or viewing the decompiled logic in Ghidra.
- **Memory Scanning:** Deploy an active agent to scan for common patching patterns, such as sequences of 0x90 (NOP) or suspicious jumps (E9) in protected memory segments.